

HelixCode Zero-Bluff Comprehensive Upgrade Plan

Project Goal

Transform HelixCode into a fully complete, zero-bluff project that exceeds all reference CLI agents in capability, with comprehensive documentation, verified testing, and guaranteed real-world usability.

Stage 1 — Deep Research & Analysis (Parallel)

1A: HelixCode Repository Deep Dive

- Clone and analyze entire HelixCode codebase
- Map all modules, services, APIs, containers, dependencies
- Identify current architecture, features, and gaps
- Document existing tests and their actual coverage

1B: HelixAgent CLI Agents Reference Analysis

- Clone and analyze cli_agents repository
- Catalog all capabilities, features, and architectural patterns
- Document what makes these agents production-ready

1C: HelixAgent Main Repository Analysis

- Clone and analyze full HelixAgent repository
- Identify advanced features, improvements, and patterns
- Map all submodules and their constitutions/CLAUDE.MD/AGENTS.MD

1D: Comparative Gap Analysis

- Side-by-side comparison: HelixCode vs HelixAgent
- Identify ALL missing features, patterns, and capabilities
- Prioritize gaps by impact and implementation complexity

Stage 2 — Constitution & Governance Framework

- Create/update Constitution.md for HelixCode
- Create/update CLAUDE.md with zero-bluff mandate
- Create/update AGENTS.md with testing requirements
- Propagate to ALL submodules

Stage 3 — Architecture & Design Documentation

- Complete system architecture diagrams
- Module interaction maps
- API specifications
- Container orchestration documentation
- Database schemas and data flow

Stage 4 — Implementation (Parallel where possible)

4A: Core Infrastructure Improvements

- Port advanced patterns from HelixAgent
- Implement missing base capabilities
- Upgrade dependency management

4B: Feature Gap Closure

- Implement all HelixAgent features missing in HelixCode
- Add production-ready patterns
- Enhanced error handling and observability

4C: Container & DevOps Alignment

- Update Docker configurations
- Add docker-compose orchestration
- Implement health checks and monitoring

Stage 5 — Anti-Bluff Testing Strategy

- Design comprehensive test pyramid
- Create integration tests that verify REAL functionality
- Add end-to-end challenge scenarios
- Implement contract tests for all APIs
- Add performance and load tests
- Create verification harnesses that fail if features don't actually work

Stage 6 — Documentation & Guides

- Step-by-step setup guides
- Developer onboarding documentation
- API usage guides with working examples
- Troubleshooting guides
- Architecture decision records (ADRs)

Stage 7 — Final Assembly & Validation

- Run complete test suite
- Execute all challenges
- Verify all documentation examples work
- Final cross-reference against reference projects
- Generate final report

Skills to Load

- Stage 1: **deep-research-swarm** (investigation)
- Stage 3/6: **report-writing** (documentation)

- Stage 4: `vibecoding-general-swarm` (implementation)
- Stage 7: `docx` (final document generation)

Success Criteria

1. All tests pass AND verify real functionality
2. All challenges pass AND prove end-to-end usability
3. All documented features are executable and working
4. No mock-only features that can't be used in production
5. Exceeds reference CLI agent capabilities
6. Complete documentation with working examples
7. Constitution/CLAUDE.MD/AGENTS.MD in all submodules